Darren Lee
CSE13S

Assignment 4: The Circumnavigations of Denver Long
Design Document

Purpose

The purpose of this assignment is to implement a depth-first search to find the Hamiltonian paths of a given graph to solve the Traveling Salesman Problem. A Hamiltonian path visits each vertex in a graph exactly once, and the solutions to the Traveling Salesman Problem are the shortest of these Hamiltonian paths. To accomplish this, I will implement a graph ADT, a path ADT, and a stack ADT.

Layout/Structure

- Graphs will be represented with adjacency matrices.
- The number of rows and columns of the n x n matrices will be determined by the number of vertices in the graph.
- The weights of the edges, which add up to the length of the path, are the values at the indices of the matrix.
- Mi,j=k represents a weight of k for the directed edge from vertex i to vertex j.
- If the matrix is undirected, there are paths going both ways from the vertices and the weights are reflected across the diagonal.

- Paths are represented by their vertices and lengths.

- Stacks have a top, a capacity, and items.

- vertices.h defines macros for the vertices
- graph.h specifies the interface to the graph ADT
- graph.c implements the graph ADT
- stack.h specifies the interface to the stack ADT.
- stack.c implements the stack ADT.
- path.h specifies the interface to the path ADT.
- path.c implements the path ADT.
- tsp.c contains main() and any helper functions

- Makefile
    - Compiler is clang
    - Has flags -Wall -Wextra -Werror -Wpedantic
    - Make and make all build the program
    - Make clean removes any files generated by the compiler
    - Make format formats .c and .h files using the given clang-format file

Functionality

- Command line options for tsp are parsed with getopt()
- The following command line options are supported:
    - -h: Prints out a help message describing the purpose of the graph and the command-line options it accepts, exiting the program afterwards
    - -v: Enables verbose printing. The program will print out all Hamiltonian paths found and the total number of recursive calls to dfs().
    - -u: Specifies the graph to be undirected
    - -i infile: Specify the input file path containing the cities and edges of a graph. If not specified, the default input is stdin
    - -o outfile: Specify the output file path to print to. If not specified, the default output is stdout
- The tsp program scans the number of cities/vertices, n, in the graph
- If the number of vertices is greater than the macro VERTICES (26), then the program prints an error message.
- The tsp program then uses fgets() to read the next n lines. If there is a malformed line, an error is printed and the program is exited.
- A new graph is created, and it is undirected if -u options is selected.
- The program then uses fscanf() until the end of the file is reached. Each edge is added to the graph. If there is a malformed line, an error is printed and the program is exited.
- Two paths are created, one for the current path and one for the shortest path.
- Starting from the vertex defined by the macro START_VERTEX in vertices.h, the program performs a depth-first search on the graph to find the shortest Hamiltonian path.
- Once the search is complete, the program prints out the length of the shortest path found, the path itself , and the number of calls to dfs().
- If the -v option was selected, the program prints out all Hamiltonian paths that were found.

Pseudocode

graph.c

Code for graph struct provided by Professor Long in assignment pdf

```
Graph struct{
        vertices
        Bool undirected
        Bool visited[VERTICES]
        matrix[VERTICES][VERTICES]
}

Graph *graph_create(vertices, bool undirected):
```

```
        Graph *G =(Graph *) malloc(Sizeof(*Graph))
        If (G):
                G->undirected =undirected
                G->vertices=(int *) calloc(vertices,sizeof(int))
                If (!G->vertices):
                        free(G)
                        G=NULL
                For i in range G->vertices:
                        G->visited[i]=false
                        For j in range G->vertices:
                                G->matrix[i][j]=0
        Return G

graph_delete(Graph **G):
        If (*G and (*G)->vertices):
                For i in range G->vertices:
                        For j in range G->vertices:
                                free((*G)->matrix[i][j])
                free((*G)->undirected)
                free((*G)->visited)
                free((*G)->vertices)
                free(*G)
                *G=NULL

graph_vertices(Graph *G):
        Return G->vertices

graph_add_edge(Graph *G, i, j, k):
        If (!(i<G->vertices) or !(j<G->vertices)):
                Return false
        G->matrix[i][j]=k
        If (G->undirected):
                G->matrix[i][j]=k
        Return true

graph_has_edge(Graph *G, i, j):
        If (i<G->vertices and j<G->vertices and G->matrix[i][j])
                Return true
        Return false

graph_edge_weight(Graph *G, i, j):
        If (!(i<G->vertices) or !(j<G->vertices) or !G->matrix[i][j]):
                Return 0
        Return G->matrix[i][j]
```

```
graph_visited(Graph *G, uint32_t v):
        Return G->visited[v]

graph_mark_visited(Graph *G, v):
        If (v<G->vertices):
                G->visited[v]=true

graph_mark_unvisited(Graph *G, v):
        If (v<G->vertices):
                G->visited[v]=false

graph_print(Graph *G):
        For i in range G->vertices:
                For j in range G->vertices:
                        If (G->matrix[i][j]):
                                Print(G->matrix[i][j])
                                Print new line
```

Path.c

Code for path struct provided by Professor Long in assignment pdf

```
Path struct{
        Stack *vertices
        int length
}

Path *path_create(void):
        Path *p=(Path *) malloc(sizeof(Path))
        If (p):
                p->vertices=stack_create(VERTICES)
                p->length=0
        Return p

path_delete(Path **p):
        If (*p and (*p)->vertices):
                free((*p)->vertices)
                free(*p)
                (*p)=NULL

path_push_vertex(Path *p, v, Graph *G):
        Declare Int y
        Declare Int *x=&y
        If (stack_push(p->vertices,v)):
```

```
                p->length+=G->matrix[stack_peek(p->vertices,x)][v]
                Return true
        Return false

path_pop_vertex(Path *p, *v, Graph *G):
        Declare int y
        Declare int *x=&y
        If (stack_pop(p->vertices,v)):
                p->length-=G->matrix[stack_peek(p->vertices,x)][v]
        Return false

path_vertices(Path *p):
        Return p->stack_size(p->vertices)

path_length(Path *p):
        Return p->length

path_copy(Path *dst, Path *src):
        If (dst):
                dst->vertices=src->vertices
                dst->length=src->length

path_print(Path *p, FILE *outfile, char *cities[]):
        stack_print(p->vertices, outfile, cities)
```

stack.c

Stack structure code provided by Professor Long in assignment pdf

```
struct Stack {
        Int top
        Int capacity
        Int *items
}

Stack *stack_create(capacity):
        Stack *s=(Stack *) malloc(sizeof(Stack))
        If (s):
                s->top=0
                s->capacity=capacity
                s->items=(int *) calloc(capacity,sizeof(int))
                if(!s->items):
                        free(s)
                        s=null
        Return s
```

```
stack_delete(Stack **s):
        if(*s && (*s)->items):
                free((*s)->items)
                free (*s)
                *s=null

stack_empty(Stack *s):
        Return !(s->top)

stack_full(Stack *s):
        s->top==s->capacity

stack_size(Stack *s):
        Return s->top

stack_push(Stack *s, x)
        if(stack_full(s)):
                Return false
        (s->items)[s->top]=x
        s->top++
        Return true

stack_peek(Stack *s, *x):
        Declare int y
        If (stack_pop(s,x)):
                y=*x
                stack_push(s,y)
                Return true
        Return false

stack_pop(Stack *s, *x)
        if(stack_empty(s)):
                Return false
        s->top--
        *x=s->items[s->top]
        Return true

stack_copy(Stack *dst, Stack *src)
        If (dst):
                dst->items=src->items
                dst->top=src->top
```

Code provided by Professor Long in assignment pdf

```
stack_print(Stack *s, FILE *outfile, char *cities[]):
```

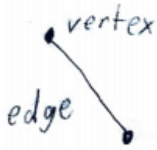
```
For i in range s->top:
        fprintf(outfile, <cities[s->items[i]]>)
        If (i+1 != s->top):
                fprintf(outfile, " -> ");
fprintf(outfile, "\n")
```

DFS

Pseudocode for depth-first search provided by Professor Long in assignment pdf

```
Procedure DFS(G,v):
        Label v as visited
        For all edges from v to w in G.adjacentEdges(v) do
                If vertex w is not labeled as visited then
                        Recursively call DFS(G,w)
        Label v as unvisited
```

Draft Work



$\langle v_i, v_j \rangle$

There is an edge connecting v_i and v_j.

$M_{2,1} = w_1$

$M_{3,4} = w_2$

$$\begin{array}{cc} & \text{columns} \\ & \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \text{rows} \begin{array}{c} 1 \\ 2 \\ 3 \\ 4 \end{array} & \begin{bmatrix} & & & \\ w_1 & & & \\ & & & w_2 \\ & & & \end{bmatrix} \end{array}$$

0 is an invalid weight/edge.

We use undirected edges by mirroring weights.

$M_{x,y} = M_{y,x}$

worst case is $O(n!)$, so largest number of vertices for this assignment is 26.

add_edge matrix [row][column] = weight